DevSync

رواد مصر الرقمية

GRADUATION PROJECT

Literature Review.

DevSync: Secure AI Project Collaborator
Feedback, improvements and grading criteria

TEAM OF 5

Donia Alhosin
Bishay Talaat
Yousef Mohy
Emad Hany
Saga Maher

REPORT DURATION

9 Oct 2026 - 16 Oct 2026

1 week

Report map

This report covers the four required components of the Literature Review deliverable. Parts 2 and 4 contain fields for the lecturer to complete; they are left blank on purpose.

Required component	Where to find it
Literature review: the research and existing work behind DevSync	Part 1, sections 1 to 8
Feedback & Evaluation: lecturer's assessment of the project	Part 2, sections 9 and 10
Suggested Improvements: areas where the project can be enhanced	Part 3, sections 11 and 12
Final Grading Criteria: breakdown of marks for documentation, implementation, testing and presentation	Part 4, sections 13 and 14

PART 1

Literature Review

1. Purpose and scope

This review explains the ideas and existing work that DevSync builds on. It answers three questions:

- Why is it hard for developers to find open-source projects that fit them?
- What do existing tools offer, and what do they leave out?
- Which recommendation, AI and engineering methods are suitable for a small Flutter and Firebase app, and how should the AI matching be evaluated?

> **Note on sources.** Section 9 lists the sources used. Please check each reference (authors, year, venue, DOI) against the original before final submission, and add any sources recommended by the supervisor. Platform descriptions reflect our reading in Oct 2026 and should be re-checked.

2. Why newcomers struggle in open source

Studies of newcomers to open-source software describe a set of barriers: finding a project and a first task, understanding the codebase and process, getting help from the community, and having the right technical

background. A systematic literature review by Steinmacher and colleagues (2015) collected these barriers. One of them, finding a suitable project and a way to start, is the problem DevSync targets.

Maintainers face the opposite problem: they need contributors with matching skills, but contributors are hard to find and assess. A tool that shows **fit** on both sides can reduce wasted effort for both.

3. Existing platforms and tools

Several tools help developers discover projects. They are useful, but they mostly filter by label, language or topic, not by the person's actual activity.

Tool	What it offers	Limits for our problem	How DevSync differs
GitHub Explore, topics and the 'good first issue' label	Browse trending repositories and filter issues that maintainers marked as beginner friendly	Filters by label or topic, not by the person's skills or history. No fit score and no request flow.	Scores each project against the user's GitHub activity and explains the score
Up For Grabs	Community-curated list of projects with beginner-friendly tasks	Static list that depends on project owners tagging tasks. No personal ranking.	Personal ranked feed that improves with like/skip feedback
CodeTriage	Lets developers subscribe to open issues of chosen repositories and receive them by email	User must already know which repositories to follow	Helps the user discover repositories in the first place
Search engines for good-first-issue lists	Filter issues by language and label	Issue-level only, no collaboration conversation	Starts with project fit and adds collaboration requests with real-time alerts

4. Recommendation approaches

Recommender systems are usually grouped into three families (Ricci et al., 2015):

- **Content-based:** match item features (languages, topics, difficulty) to a user profile. Works for new users who have a profile but no history.
- **Collaborative filtering:** recommend what similar users liked. Needs many users and past interactions.
- **Hybrid:** combine both to balance their weaknesses.

A new app has the **cold-start problem**: few users and no history. DevSync therefore starts with a content-based approach driven by GitHub data and declared skills, and stores like and skip signals so a hybrid approach becomes possible once there is enough data.

5. Large language models for matching

Large language models can compare a structured profile with a project description and produce a score and a short explanation without training a custom model. Prompting techniques are central to this:

- **Few-shot prompting** shows the model a few examples of the wanted input and output (Brown et al., 2020). We use it to fix the output format (score and reason).

- **Step-by-step reasoning** can improve results on complex tasks (Wei et al., 2022). DevSync asks only for a short reason to keep cost and latency low.
- **Model-as-judge evaluation** uses a strong model to grade outputs at scale, but it has known biases (Zheng et al., 2023). We therefore combine it with a fixed evaluation set and real user feedback.

How DevSync evaluates prompts

Step	What we do	Evidence produced
1. Fixed evaluation set	Sample profiles and projects with expected relevance labels agreed by the team	Evaluation set in the repository
2. Score prompt versions	Run each prompt version on the set and compare relevance	Relevance score per version (KPI: +15% first to final)
3. Real feedback	Compare scores with like/skip feedback from pilot users	Share of top-5 matches rated relevant (KPI: 50% or more)
4. Keep the best	Keep the best prompt version and document why	AI documentation

6. Mobile engineering and security practice

- **Backend as a service.** Firebase offers authentication, a document database and push messaging, which suits a small team with a fixed deadline. Firestore security rules decide who can read and write each document.
- **Mobile security.** The OWASP Mobile Application Security Verification Standard (MASVS) gives a checklist of areas such as storage, network, authentication and code quality. Our security checklist follows these areas: no secrets in the repository, HTTPS only, least-privilege permissions and obfuscated release builds.
- **Obfuscation.** Android build tooling can shrink and obfuscate code; it makes reverse engineering harder but is not a replacement for server-side checks.
- **Agile Scrum.** The Scrum Guide (Schwaber and Sutherland, 2020) describes short sprints, planning, review and retrospective. We use one-week sprints, which fit the six-week schedule.

7. Gaps and how DevSync responds

Gap found in the literature or tools	DevSync response
Discovery tools rely on labels and topics, not on the person	Profile built from GitHub activity and declared skills
No explanation of why a project fits	Every match shows a score and a short reason
No feedback loop to improve results	Like and skip signals feed prompt evaluation
Contact between contributor and maintainer happens elsewhere	Collaboration requests with real-time alerts inside the app
Prompt quality is rarely measured in small projects	Fixed evaluation set plus pilot feedback, with KPIs
Security is often left for the end	Security checklist, rules review and an obfuscated build tested by 12 Nov

8. Summary

The literature shows that finding a project is a real barrier for newcomers, that current tools do not rank projects for the individual, and that content-based recommendation with LLM scoring is a practical start when there is no history. DevSync combines these ideas and adds a measured prompt-improvement loop, which is where it can show its own contribution.

9. References

[1] Brown, T. B. et al. (2020). Language Models are Few-Shot Learners. Advances in Neural Information Processing Systems (NeurIPS) 33.

[2] OWASP Foundation. Mobile Application Security Verification Standard (MASVS). owasp.org.

[3] Ricci, F., Rokach, L. and Shapira, B. (eds.) (2015). Recommender Systems Handbook, 2nd edition. Springer.

[4] Schwaber, K. and Sutherland, J. (2020). The Scrum Guide. scrumguides.org.

[5] Steinmacher, I., Graciotto Silva, M. A., Gerosa, M. A. and Redmiles, D. F. (2015). A systematic literature review on the barriers faced by newcomers to open source software projects. Information and Software Technology, 59.

[6] Wei, J. et al. (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. NeurIPS 35.

[7] Zheng, L. et al. (2023). Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. NeurIPS 2023, Datasets and Benchmarks.

[8] Official documentation: Flutter, Firebase (Authentication, Cloud Firestore, Cloud Messaging, Security Rules) and Android developer guides. Accessed Oct 2026.

[9] Platform pages: GitHub Explore and topics, Up For Grabs (up-for-grabs.net), CodeTriage (codetriage.com). Accessed Oct 2026.

PART 2

Feedback & Evaluation

10. Lecturer's assessment

To be completed by the lecturer or supervisor. Ratings use a 1-5 scale (1 = needs major work, 5 = excellent).

Field	Entry
Lecturer / supervisor	_____
Date of review	_____
Version reviewed	_____

Area assessed	What to look at	Rating (1-5)	Comments
Problem and idea	Clear problem, realistic scope, value for users	____	
Planning and management	Plan, roles, risks, KPIs, evidence of following the plan	____	
Requirements	Stakeholders, stories, functional and non-functional requirements	____	
Technical quality	Architecture, Firebase use, AI matching, code quality	____	
Security and release	Permissions, rules, obfuscation, release builds	____	
Testing	Test coverage, bug list, prompt evaluation	____	
Documentation	README, technical documentation, report quality	____	
Presentation and demo	Clarity, working demo, answers to questions	____	

Overall comments	
Main strengths	
Main concerns	
Overall result	

11. Team self-evaluation plan

The team will collect evidence for the assessment during the project, so the lecturer can see results and not only plans.

Evidence	Collected from	When
Weekly progress summary and sprint report	Jira sprint reports, supervisor update	Every week
Demo of working features	Sprint review call	End of each week
Test and security report	CI results, bug list, security checklist	12 Nov, final update 30 Nov
Prompt improvement results	Evaluation set results by prompt version	12 Nov and 30 Nov
Pilot results (adoption, match quality, requests)	Firebase counts and like/skip feedback	Late Nov
Git history and pull request review record	GitHub	Continuous

PART 3

Suggested Improvements

12. Improvements identified by the team

These come from our own review of the project. Items marked **Now** can still fit before 30 Nov; items marked **Future** are ideas to make DevSync stronger after the deadline.

Area	Improvement	Benefit	Effort	When
Security	Keep the LLM API key in a Cloud Function secret, not in the app	Prevents key theft and cost abuse	S	Now
AI	Fallback ranking from skills only when the LLM is slow or fails	App stays useful during outages	S	Now
AI	Cache match results and limit calls per user	Lower cost and faster feed	S	Now
Quality	Run the evaluation set automatically in CI	Every prompt change is measured	M	Future
AI	Embedding-based pre-filter, then LLM ranking of the top candidates	Cheaper and faster at larger scale	M	Future
AI	Issue-level matching (good first issues) and skill-gap suggestions	Tells users which task to start and what to learn	M	Future
AI	Remote Config for prompt versions with A/B tests	Compare prompts without a new release	M	Future
Product	Maintainer search for contributors, in-app chat and endorsements	Completes the two-sided marketplace	L	Future
Product	Arabic localization with right-to-left layout	Reaches more local developers	M	Future
Product	Onboarding tour and shareable profile card	Higher completion and word of mouth	S	Future
Trust	Report and block users, GitHub-verified profiles, Firebase App Check	Safer community and protected backend	M	Future
Platform	Web platform on the same Firebase backend, then iOS	Wider reach	L	Future
Business	Real Pro plan with in-app payments	Turns the Pro flavor into a business model	L	Future

Effort: S = under 1 day, M = 2-4 days, L = more than 4 days (planning estimates).

13. Lecturer suggestions and action tracker

Fill in during or after the review. The project manager copies each action into Jira.

Suggestion from lecturer	Action we will take	Owner	Due	Status

PART 4

Final Grading Criteria

14. Marks breakdown

The marks for each criterion are set by the course. Fill in the available marks from the course handbook; the evidence column shows what the team will present for each criterion.

Criterion	What is assessed	Evidence from DevSync	Marks
Documentation	Planning report, literature review, requirements, technical documentation, README, clean Git history	Three reports, README, architecture and data model docs, AI and prompt documentation	__ / __
Implementation	Working features, use of Flutter, Firebase Auth, FCM, animations, flavors, code quality	Working app with must-have features, Free and Pro flavors, pull request history	__ / __
Testing	Unit, widget and integration tests, manual testing, security review, prompt evaluation	Test and security report, CI results, bug list, evaluation set results	__ / __
Presentation	Clear story, live demo, teamwork, answers to questions	Final presentation and demo on 30 Nov, with every member presenting a slice	__ / __
Total			__ / __

PART 5

Future Enhancements

16. From a task board to a full task management system

The 30 Nov scope includes only an optional shared board for two accepted collaborators. The next step is to grow it into a complete task management system inside DevSync, so that matching, requests, collaboration and delivery all happen in one place. Each accepted collaboration becomes a workspace with members, roles, tasks, sprints, alerts and reports, connected to GitHub and the tools teams already use.

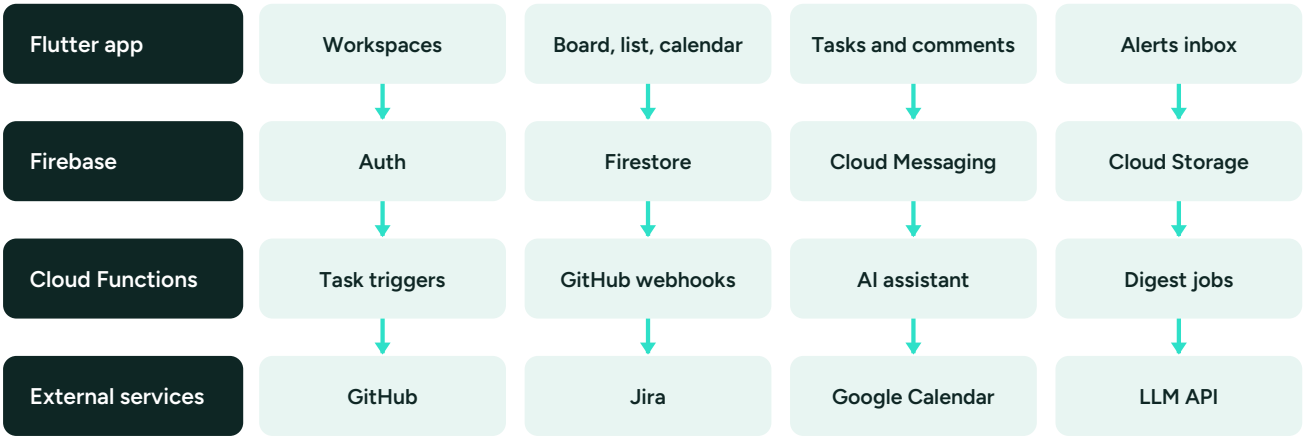
> **Why it fits DevSync.** The literature shows that contact between contributor and maintainer happens outside the tool (section 7). A built-in task system keeps the work, the people and the progress together, and gives the team new data for the adoption and collaboration KPIs.

Capabilities of the full system. Effort: S = under 1 day, M = 2-4 days, L = more than 4 days (planning estimates). Next = first phase after 30 Nov; Later = following phases.

Capability	What it adds	Integration	Effort	When
Project workspaces	One workspace per accepted collaboration, with members, roles (owner, maintainer, contributor) and an activity feed	Firestore, security rules	M	Next
Full task model	Title, description, assignee, priority, labels, due date, subtasks, checklists, comments and a status flow (To do, In progress, In review, Done)	Firestore	M	Next
Board, list and calendar views	Drag and drop, filters and saved views so each person can work their own way	Flutter UI	M	Next
GitHub integration	Link a task to an issue, branch or pull request; move the task when a pull request is merged; import good first issues as tasks	GitHub API and webhooks via Cloud Functions	L	Next
Smart notifications	Alerts for assignment, mentions, due dates and a daily digest, with per-user settings and an in-app inbox	FCM, Cloud Functions	M	Next
Roles, permissions and audit log	Who can create, assign, edit or delete tasks, with a record of every change	Firestore rules	M	Next
Sprints and backlog	Plan sprints, estimate with story points, see a burndown chart	Firestore, Cloud Functions	L	Later
AI task assistant	Split an issue into tasks, suggest effort and assignee from the skill match, write a weekly progress summary	LLM through a Cloud Function, existing prompt pipeline	M	Later
Jira and Trello import and sync	Bring in existing boards and keep them in sync for teams that already use them	Jira Cloud REST API	L	Later
Live collaboration	Real-time updates, @mentions, file attachments and presence on a task	Firestore listeners, Cloud Storage	M	Later
Dashboards and reports	Velocity, cycle time and contributor activity linked to the project KPIs	Firebase Analytics, BigQuery	M	Later
Calendar sync and offline mode	Show due dates in Google Calendar; keep working offline and sync later	Google Calendar API, local cache	L	Later

17. Integration architecture

The system reuses the stack already chosen for DevSync, so no separate server is needed. The app talks only to Firebase; Cloud Functions handle everything that needs a secret key or an outside service.



Arrows show the direction of requests. Webhook secrets, the LLM key and Jira tokens stay in Cloud Function secrets, never in the app or the repository.

18. Phased roadmap

Phase	Timing (proposed)	Deliverables	Done when
Phase 1: Foundation	Dec 2026	Workspaces, full task model, roles and permissions, task alerts	Two collaborators can plan and finish a project from a workspace
Phase 2: Connected work	Jan-Feb 2027	Board, list and calendar views, comments, GitHub issue and pull request links	A merged pull request moves its task automatically
Phase 3: Planning and insight	Mar-Apr 2027	Sprints and backlog, dashboards, AI task assistant, Jira import	A team can run a sprint and read its report in the app
Phase 4: Reach	From May 2027	Live collaboration, calendar sync, offline mode, web version	The same workspace works on phone and desktop

Timing is a planning estimate and depends on the results of the 30 Nov delivery and the pilot.

19. Success measures and risks

Measure or risk	Target or response	How it is checked
Workspace adoption	At least 50% of accepted collaborations create a task within 24 hours	Firestore counts
Task completion	At least 60% of tasks reach Done within their due date	Task records, dashboard
GitHub sync accuracy	At least 95% of merged pull requests update the linked task	Webhook logs
Scope growth	Build in phases; each phase ships on its own and keeps the core stable	Jira sprint reports
Notification overload	Per-user settings, a daily digest and quiet hours	User feedback
Permissions and data leaks	Rules unit tests for every role; audit log for changes	Rules tests, review
Integration limits and cost	Cache GitHub data, batch calls, budget alerts, App Check and rate limits	Usage dashboards